h e p i a

Haute école du paysage, d'ingénierie
et d'architecture de Genève

OCR

Vision numérique

2024-06-07

Adrian Spycher
Haute école du paysage, d'ingénierie et d'architecture

I. Introduction

Le but de ce projet est d'implémenter une application web permettant de faire de la reconnaissance de caractères (OCR). Dans notre cas, seuls les chiffres seront considérés. Ce projet se découpe en trois parties distinctes, le frontend web, l'API et le backend. Chacune de ces étapes est détaillée dans la suite du rapport.

II. Implémentation

II.1. Frontend web

N'ayant pas de design donné, nous avons opté pour implémenter un frontend avec une vue très simple, composée uniquement d'un canvas et de progress bar HTML.

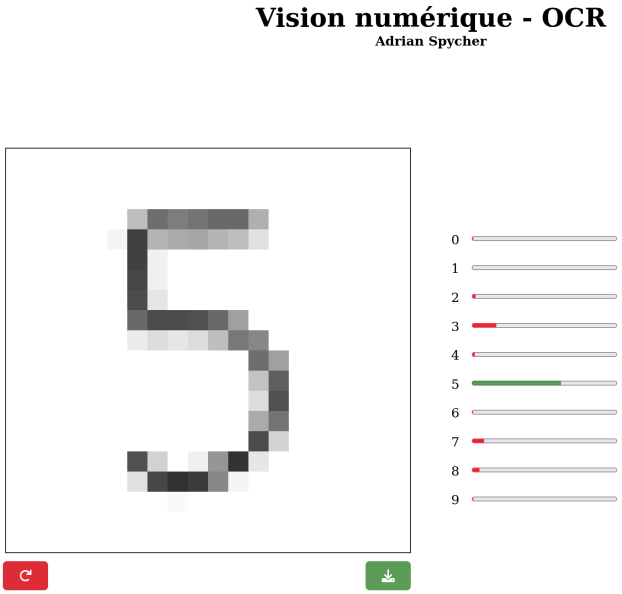


Fig. 1 — Frontend du site web

Pour correctement afficher le canvas, nous avons eu quelques problèmes, car, dans notre cas, il fallait réduire la taille du canvas à 20×20 pixels. Sauf qu'en faisant cela, il devient impossible de dessiner dessus. Alors, nous l'avons créé d'une taille fixe de 20×20 pixels, tout en allant modifier sa taille de rendu, via du CSS, à 500×500 pixels.

Pour envoyer notre canvas sous forme d'image à notre api, via `post-canvas`, nous avons utilisé la fonction `await fetch` de JavaScript. L'envoie de l'image à l'api nous renvois un tableau contenant les différentes prédictions selon chaque chiffre. Celles-ci sont utilisées pour rafraîchirr les progress bar.

`script.js`

```
const response = await fetch("/post-canvas", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
    },

    body: JSON.stringify({
        pixels: Array.from(pixels_gray),
        width: canvas.width,
        height: canvas.height,
    }),
});

if (!response.ok) {
    throw new Error("Network error " + response.statusText);
}

const data = await response.json();
const prediction = data.prediction[0];
const number = data.number;

updateProgress(prediction, number);
```

Liste 1 — Code JavaScript effectuant un appel à l'API

II.2. Backend

Dataset

Pour ce projet, nous étions très tentés d'utiliser des datasets déjà faits tels que `MNIST`. Malheureusement, pour ce projet, nous devons créer notre propre dataset. Pour cela, via le frontend, nous avons dessiné 20 images pour chaque chiffre que nous avons enregistré dans un fichier compressé nommé `numbers_20.tar.xz`.

Lors de la création du dataset, nous avons fait attention à plusieurs choses afin d'avoir plus d'irrégularité dans nos images, telles que :

- dessiner les chiffres à des positions différentes
- utiliser différentes « police » d'écriture
- varier les tailles des chiffres

Entraîner un dataset ayant de telles variétés permet de rapprocher le modèle de la réalité et de le rendre fiable même lors du changement de dessinateur. Néanmoins, utiliser ces variantes nous a fait douter du nombre d'images par chiffre.

En effet, nous pensions que 20 images par chiffres ne seraient pas suffisantes pour avoir un modèle fiable. Donc, nous avons aussi créé une version de 50 images par chiffre qui se nomme `numbers_50.tar.xz` aussi disponible sur le Git. Voici quelques exemples d'images de notre ensemble de données :



Fig. 2 — Frontend du site web

Models

Afin de correctement entrainer le modèle, nous avons tout d'abord normalisé le dataset d'entrée. Dans cette étape, nous avons effectué deux changements aux images :

- un seuillage
- un décalage en haut à gauche du chiffre par rapport à l'image

Le seuillage était nécessaire dans le but de n'avoir que des 1 et des 0 dans l'image. Comme *threshold* nous avons choisi 220, qui est une valeur arbitraire. Pour le décalage, nous voulions restreindre au maximum les variantes que pouvait amener une image.

Les paramètres d'entrainement ont été résumés dans la table ci-dessous.

Paramètres	**Valeur**	**Paramètres**	**Valeur**
taille de test	10 %	optimiseur	Adam
dimensions d'entrée	400 (20×20)	taux d'apprentissage	1e-4
dimensions de sortie	10 classes	taille des lots	32
couches cachées	[512, 256, 128]	nombre d'époques	30
fonction activation	softmax		

Tableau 1 — Paramètres d'entrainement

II.3. API

L'API que nous avons implémentée est faite en Python, à l'aide de `FastAPI`. La plus grande partie de notre API se situe dans la route `post-canvas`. C'est là où nous récupérons et effectuons la prédiction d'une image donnée.

Afin d'effectuer la prédiction, nous avons, tout d'abords, normalisé l'image pour qu'elle soit cohérente par rapport à notre modèle. Ensuite, le modèle généré au préalable nous permet de prédire de quel chiffre il s'agit. Enfin, nous renvoyons un tableau contenant le résultat de la prédiction.

api.py

```
  # - receive data -

  # [...]

  grayscale_array = np.array(pixels, dtype=np.uint8).reshape((height, width))
  img_bin = np.where(grayscale_array < 220, 1, 0)
  shifted_img_bin = shift_to_top_left(img_bin)

  # - make prediction -

  try:
      input_data = np.expand_dims(shifted_img_bin, axis=0)
      prediction = model.predict(input_data).tolist()

     return JSONResponse(content={"message": "Pixel data received", "prediction":
prediction, "number": np.argmax(prediction).tolist()})

  except Exception as e:
      return JSONResponse(content={"error": str(e)}, status_code=500)
```

Liste 2 — HTML permettant d'instancier une canvas

III. Résultats

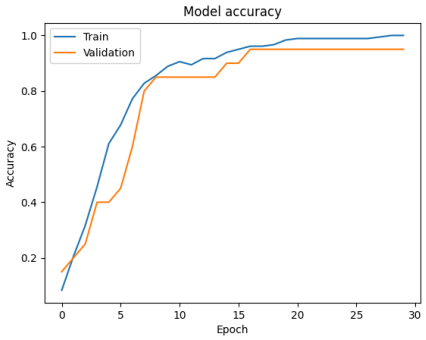


Fig. 3 — Analyse de la précision du modèle

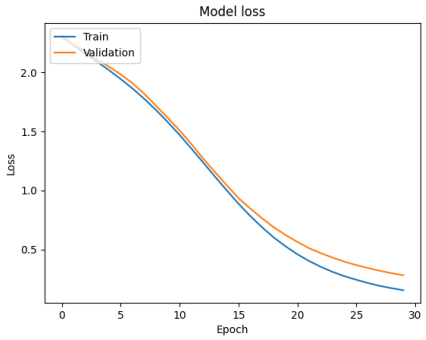


Fig. 4 — Analyse de la perte du modèle

Dans l'analyse de la précision, on constate que lors de l'entrainement, le modèle croît rapidement, atteignant vite la précision de 1 (100 %). De la même manière, la courbe de validation croit rapidement, mais on voit qu'elle atteint un plateau à 0.9 (90 %). Ce plateau suggère un léger *overfiting*.

Dans l'analyse de la perte du modèle, on constate que l'on atteint déjà des valeurs très basses après environ 20 époques. La courbe de validation suit une tendance similaire, mais laissant un écart de 0,2 à la fin de l'entrainement.

Malgré les très bons résultats que nous pouvons voir ici, lors d'un essai sur le site internet. On constate que l'exactitude est bonne, mais n'est pas suffisante. Il arrive parfois au modèle de se tromper, ce qui peut s'expliquer du fait du peu de nombre d'images ou bien de la différence d'écriture de chacun.

IV. Conclusion

Dans ce projet de reconnaissance de chiffres, nous avons entraîné un modèle sur un ensemble de donnée contenant 20 images par chiffres. Nous avons constaté que les résultats étaient assez concluants malgré la taille de l'ensemble de donnée.

Afin d'améliorer l'efficacité de ce modèle, il aurait été bénéfique d'explorer les méthodes de réseau neuronal convolutif qui sont généralement utilisées lors de l'OCR, entre autres, en raison de leur efficacité. Lors de la normalisation, une bonne idée aurait été aussi de normer la taille des chiffres dans les images. Les images étant de petites tailles, ceci peut être assez complexe.